

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

MODELO 1

QUESTÃO 01 – VALOR 0,25 PONTO

Você está construindo um módulo de seleção adaptativa de regras de firewall baseando-se no Problema da Mochila e possui um conjunto de regras candidatas, em que cada regra possui um Custo de Processamento (CPU) e um Valor de Risco Mitigado. A capacidade total de CPU do servidor é limitada em 100 unidades. Se uma combinação de regras estourar esse limite, a solução é considerada inválida para o ambiente operacional.

1. Complete a função Python abaixo (`calcular_fitness`) recebendo um indivíduo binário (vetor de 0s e 1s), o vetor de custos, o vetor de valores e a capacidade máxima. A função deve aplicar uma penalização severa ($fitness = 0$) caso o custo total ultrapasse a capacidade máxima; caso contrário, deve retornar a soma do valor mitigado.
2. Explique por que a aplicação de penalidade na função de *fitness* é indispensável ao lidar com problemas de otimização sob restrição.

```
import numpy as np
```

```
def calcular_fitness(individuo, custos, valores, capacidade_maxima):
```

```
# TODO: Calcule o custo total e o valor total do individuo
```

```
# TODO: Aplique a regra de restrição e retorne o fitness correto
```

```
# Produto escalar/soma ponderada para obter o custo e o valor brutos
```

```
custo_total = np.sum(individuo * custos)
```

```
valor_total = np.sum(individuo * valores)
```

```
# Aplicação da penalidade por estouro do limite de CPU (restrição)
```

```
if custo_total > capacidade_maxima:
```

```
    return 0 # Penalização severa
```

```
    return valor_total
```

Resposta Técnica Esperada

Ao lidar com problemas de otimização sob restrição no mundo real (como limite de memória ou CPU), a penalização é indispensável para alterar a "paisagem de aptidão" (*fitness landscape*). Ela impede que soluções inviáveis dominem o processo de seleção. Sem a penalidade, indivíduos que ativassem todas as regras acumulariam a maior pontuação bruta e seriam selecionados para reprodução, propagando genes inválidos e impedindo o algoritmo de encontrar uma solução funcional.

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

QUESTÃO 02 – VALOR 0,25 PONTO

Cenário:

Um sistema de proteção de rede utiliza o algoritmo PSO (*Particle Swarm Optimization*) para calibrar dinamicamente os limiares matemáticos (*thresholds*) de sensibilidade contra ataques de negação de serviço (DDoS). Cada partícula navega no espaço contínuo buscando a combinação de parâmetros que minimize o número de falsos positivos.

1. Escreva o trecho em Python responsável por atualizar a velocidade da partícula i no instante atual, considerando a equação do PSO que combina os componentes de Inércia (w), Atração Cognitiva (c_1) e Atração Social (c_2).
2. Explique a função técnica de cada um dos três componentes (w , c_1 e c_2) na trajetória da partícula.

```
import numpy as np
```

```
def atualizar_velocidade(V_i, X_i, pbest_i, gbest, w=0.5, c1=1.5, c2=1.5):
```

```
    r1, r2 = np.random.rand(), np.random.rand()
```

```
    # Equação do PSO: Inércia + Componente Cognitivo + Componente Social
```

```
    V_nova = (w * V_i) + (c1 * r1 * (pbest_i - X_i)) + (c2 * r2 * (gbest - X_i))
```

```
    return V_nova
```

Resposta Técnica Esperada

- **Inércia (w):** Mantém a tendência e a direção do movimento anterior da partícula, promovendo a exploração livre do espaço de busca (*exploration*) e evitando paradas abruptas.
- **Atração Cognitiva (c_1):** Representa a "memória individual" da partícula, puxando-a de volta em direção ao melhor ponto que *ela própria* já visitou (p_{best}).
- **Atração Social (c_2):** Representa a "cooperação em grupo", puxando a partícula em direção ao melhor ponto global encontrado por *qualquer* integrante do enxame (g_{best}).

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

MODELO 2

QUESTÃO 01 – VALOR 0,25 PONTO

Em uma infraestrutura de telecomunicações, um algoritmo ACO (*Ant Colony Optimization*) é utilizado para descobrir a rota de menor latência entre dois *datacenters*. À medida que as “formigas virtuais” percorrem a rede, elas deixam marcas de feromônio nos enlaces. No entanto, para evitar que caminhos antigos e lentos fiquem marcados para sempre, especialmente quando a topologia muda devido a falhas, a matriz de feromônio precisa ser constantemente atualizada via evaporação e novos depósitos.

1. Escreva a função em Python `atualizar_matriz_feromonio` que recebe a matriz atual de feromônio, a taxa de evaporação (ρ), e uma lista de caminhos encontrados pelas formigas junto com seus respectivos custos de latência. A função deve aplicar a evaporação em toda a matriz e, em seguida, adicionar novo feromônio inversamente proporcional ao custo do caminho ($1.0/latencia$).
2. Explique a função biológica/computacional da evaporação do feromônio (ρ) no comportamento da colônia.

```
import numpy as np
```

```
def atualizar_matriz_feromonio(matriz_feromonio, rho, caminhos, custos_latencia):
```

```
    # TODO 1: Evaporação do feromônio em toda a matriz
```

```
    matriz_feromonio = (1.0 - rho) * matriz_feromonio
```

```
    # TODO 2: Depósito de novo feromônio proporcional à qualidade da rota
```

```
    for caminho, latencia in zip(caminhos, custos_latencia):
```

```
        deposito = 1.0 / latencia
```

```
        for i in range(len(caminho) - 1):
```

```
            u = caminho[i]
```

```
            v = caminho[i + 1]
```

```
            matriz_feromonio[u][v] += deposito
```

```
            matriz_feromonio[v][u] += deposito # Se o grafo for não-direcionado
```

```
    return matriz_feromonio
```

Resposta Técnica Esperada

A evaporação do feromônio (ρ) atua como um mecanismo de "esquecimento controlado". Biológica e computacionalmente, ela evita que a colônia fique presa a escolhas antigas ou rotas subótimas. A evaporação reduz continuamente a atratividade de caminhos menos eficientes e permite que o algoritmo continue explorando novas opções (*exploration*), ajustando-se a mudanças dinâmicas na topologia da rede (como quedas de enlaces).

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

QUESTÃO 02 – VALOR 0,25 PONTO

Um provedor de serviços em nuvem utiliza um Algoritmo Memético para alocar máquinas virtuais em servidores físicos, minimizando consumo de energia. O algoritmo AG realiza busca global no espaço de alocações, mas, antes da fase de reprodução, cada indivíduo passa por um processo de aprendizado individual (Busca Local) para tentar melhorar seu arranjo refinando a alocação nas vizinhanças imediatas.

1. Complete a função `busca_local_hill_climbing` que recebe uma solução candidato (vetor de alocação) e tenta realizar até `max_passos` de melhoria local. Em cada passo, um vizinho é gerado adicionando um pequeno ruído à solução. Se o vizinho apresentar um consumo de energia menor (*fitness*), a solução atual é substituída. (escreva só a parte do TODO)
2. Explique a diferença entre a estratégia de busca de um AG Puro e um Algoritmo Memético.

```
import numpy as np

def calcular_energia(solucao):
    return np.sum(solucao ** 2)

def busca_local_hill_climbing(solucao_inicial, max_passos=10, tamanho_passo=0.05):
    solucao_atual = np.copy(solucao_inicial)
    custo_atual = calcular_energia(solucao_atual)

    for _ in range(max_passos):
        # TODO 1: Gera vizinho com ruído no intervalo [-tamanho_passo, tamanho_passo]

        ruído = np.random.uniform(-tamanho_passo, tamanho_passo, size=len(solucao_inicial))
        vizinho = solucao_atual + ruído

        # TODO 2: Calcula o custo do vizinho e substitui se for melhor (menor consumo)

        custo_vizinho = calcular_energia(vizinho)

        if custo_vizinho < custo_atual:
            solucao_atual = vizinho
            custo_atual = custo_vizinho

    return solucao_atual
```

Resposta Técnica Esperada

Um AG Puro utiliza apenas operadores biológicos globais (cruzamento e mutação aleatória) para evoluir a população ao longo das gerações. Já um Algoritmo Memético combina a evolução populacional global com um processo de aprendizado individual/intensificação local (*Hill Climbing*): antes da fase reprodutiva, cada indivíduo explora ativamente sua vizinhança imediata para refinar e otimizar sua própria solução localmente.

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

MODELO 3

QUESTÃO 01 – VALOR 0,25 PONTO

Em um sistema de triagem e resposta automática a incidentes cibernéticos (SOC), um Algoritmo Genético é utilizado para selecionar o conjunto ideal de regras de detecção a serem ativadas no Firewall. O objetivo é maximizar a cobertura de ameaças sem ultrapassar a capacidade limite de processamento por segundo (PPS) do appliance. Para garantir que as melhores combinações de regras sejam preservadas e propagadas entre as gerações, o algoritmo utiliza o método de seleção por torneio.

1. Escreva em Python a função `selecao_torneio` que recebe a população de indivíduos binários, o vetor com os valores de *fitness* de cada indivíduo e o tamanho do torneio k . A função deve escolher aleatoriamente k indivíduos da população e retornar aquele que possui o maior valor de *fitness*.
2. Explique o conceito de pressão seletiva no contexto da Seleção por Torneio.

```
import numpy as np
```

```
def selecao_torneio(populacao, fitnesses, k=3):
```

```
    # TODO 1: Sorteia k índices válidos aleatórios da população
```

```
    indices_sorteados = np.random.choice(len(populacao), size=k, replace=False)
```

```
    # TODO 2: Encontra qual dos k sorteados possui o maior valor de fitness
```

```
    melhor_indice = indices_sorteados[np.argmax(fitnesses[indices_sorteados])]
```

```
    return populacao[melhor_indice]
```

Resposta Técnica Esperada

A pressão seletiva é a força com que o algoritmo favorece a escolha dos indivíduos mais aptos em detrimento dos menos aptos. No método por Torneio, a pressão seletiva é diretamente controlada pelo tamanho k : quanto maior o valor de k , maior é a probabilidade de que indivíduos com *fitness* elevado estejam presentes no grupo disputante, reduzindo as chances de sobrevivência de indivíduos fracos ou medianos.

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

QUESTÃO 02 – VALOR 0,25 PONTO

Um sistema de roteamento inteligente de pacotes em redes de sensores sem fio utiliza o algoritmo PSO (*Particle Swarm Optimization*) para otimizar o consumo energético dos nós transmissores. As partículas representam diferentes configurações de potência de sinal e navegam continuamente pelo espaço de busca tentando encontrar a combinação que minimiza a perda de pacotes e o gasto de bateria.

1. Escreva em Python o trecho da função `atualizar_posicao` que recebe a posição atual da partícula X_i , sua velocidade recém-calculada V_i e os limites inferior e superior do espaço de busca. A função deve atualizar a posição da partícula e aplicar o limitador (*clamp*) para garantir que a partícula não saia das fronteiras permitidas.
2. Explique a diferença entre o comportamento de Exploration (Exploração) e Exploitation (Exploração/Intensificação) durante a busca do PSO.

```
import numpy as np
```

```
def atualizar_posicao(X_i, V_i, limite_min=-10.0, limite_max=10.0):
```

```
    # TODO 1: Atualização da posição somando o vetor de velocidade
```

```
    X_novo = X_i + V_i
```

```
    # TODO 2: Aplicação do limite (clip/clamp) no espaço de busca
```

```
    X_novo = np.clip(X_novo, limite_min, limite_max)
```

```
    return X_novo
```

Resposta Técnica Esperada

- **Exploration (Exploração):** É a capacidade do enxame de cobrir áreas amplas e variadas do espaço de busca para descobrir novas regiões promissoras.
- **Exploitation (Exploração/Intensificação):** É a capacidade de focar e refinar a busca na vizinhança local de uma boa solução já identificada.
- **Diferença/Comportamento:** *Exploration* mapeia o espaço globalmente, enquanto *Exploitation* faz os ajustes finos para encontrar o mínimo/máximo exato.

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

MODELO 4

QUESTÃO 01 – VALOR 0,25 PONTO

Em um problema de roteamento dinâmico de frotas de entrega urbana, um algoritmo ACO (*Ant Colony Optimization*) avalia a atratividade de diferentes vias de trânsito. A escolha do próximo nó (cruzamento) por uma formiga virtual depende do equilíbrio entre a quantidade de feromônio acumulado na aresta (τ) e a atratividade heurística da via (η), que é dada pelo inverso da distância/tempo de tráfego ($1.0/distancia$).

Escreva em Python a função `calcular_probabilidade_transicao` que recebe o vetor de feromônios das arestas candidatas, o vetor de distâncias até os nós vizinhos não visitados, o expoente do feromônio (α) e o expoente da heurística (β). A função deve calcular e retornar o vetor de probabilidades normalizado para a escolha do próximo nó. (complete somente os TODO)

```
import numpy as np
```

```
def calcular_probabilidade_transicao(feromonios, distancias, alpha=1.0, beta=2.0):
```

```
    # TODO 1: Calcule a atratividade heurística (inverso da distância)
```

```
    eta = 1.0 / (distancias + 1e-6) # 1e-6 evita divisão por zero
```

```
    # TODO 2: Calcule o valor não-normalizado das opções
```

```
    pesos = (feromonios ** alpha) * (eta ** beta)
```

```
    # TODO 3: Normalize o vetor para obter probabilidades (soma = 1.0)
```

```
    probabilidades = pesos / np.sum(pesos)
```

```
    return probabilidades
```

AVALIAÇÃO AAI – COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

QUESTÃO 02 – VALOR 0,25 PONTO

Em um sistema de triagem e resposta automática a incidentes cibernéticos (SOC), um Algoritmo Genético é utilizado para selecionar o conjunto ideal de regras de detecção a serem ativadas no Firewall. O objetivo é maximizar a cobertura de ameaças sem ultrapassar a capacidade limite de processamento por segundo (PPS) do appliance. Para garantir que as melhores combinações de regras sejam preservadas e propagadas entre as gerações, o algoritmo utiliza o método de seleção por torneio.

1. Escreva em Python a função `selecao_torneio` que recebe a população de indivíduos binários, o vetor com os valores de *fitness* de cada indivíduo e o tamanho do torneio k . A função deve escolher aleatoriamente k indivíduos da população e retornar aquele que possui o maior valor de *fitness*.
2. Explique o conceito de pressão seletiva no contexto da Seleção por Torneio.

```
import numpy as np
```

```
def selecao_torneio(populacao, fitnesses, k=3):
```

```
    # TODO 1: Seleciona k índices aleatórios válidos sem repetição
```

```
    indices_sorteados = np.random.choice(len(populacao), size=k, replace=False)
```

```
    # TODO 2: Identifica e retorna o indivíduo de maior fitness entre os k sorteados
```

```
    melhor_indice = indices_sorteados[np.argmax(fitnesses[indices_sorteados])]
```

```
    return populacao[melhor_indice]
```

Resposta Técnica Esperada

A pressão seletiva define o grau de vantagem dado aos indivíduos de alto desempenho no processo evolutivo. Na Seleção por Torneio, o parâmetro k ajusta diretamente essa pressão: valores maiores de k aumentam a exigência e a competição, garantindo a passagem apenas dos melhores indivíduos, enquanto valores menores de k permitem que indivíduos mais fracos também tenham uma oportunidade aleatória de reprodução, preservando a diversidade genética.